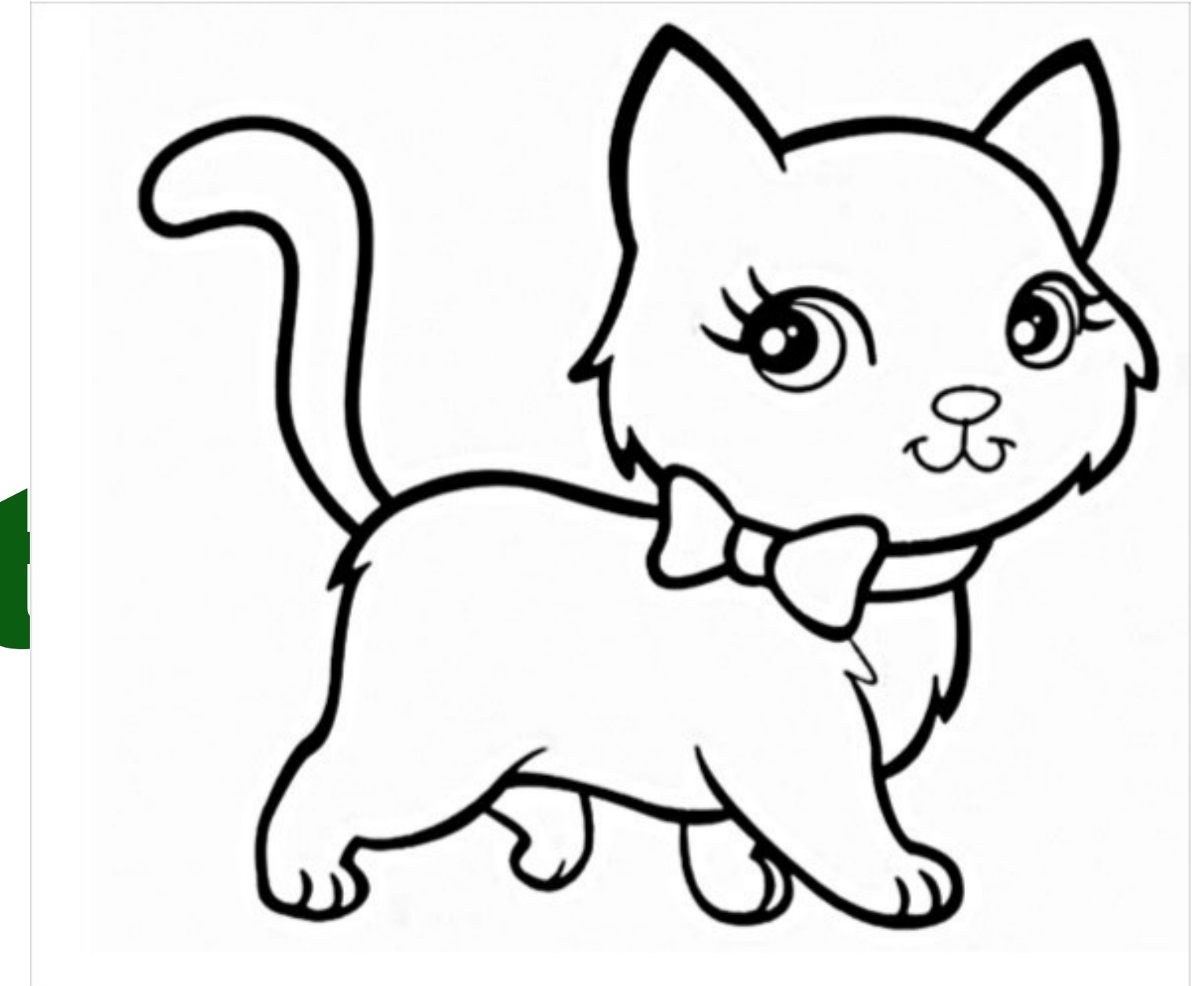
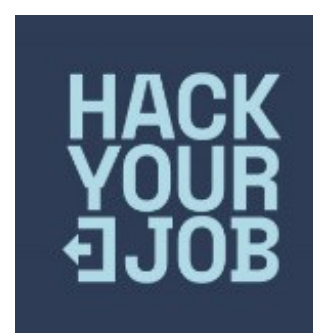


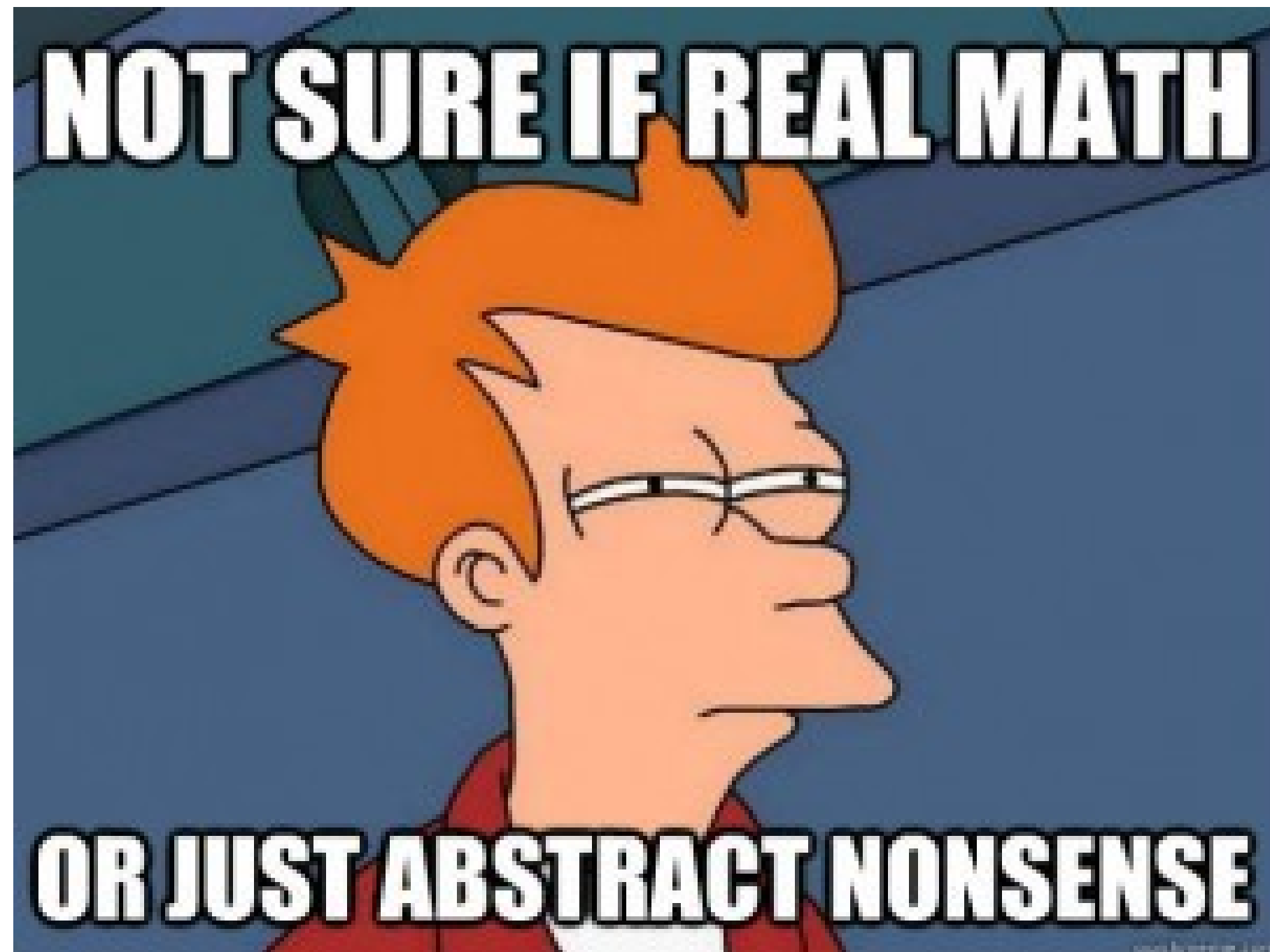
Category Theory: You already know it



Romain Berthon @romaintrm
Emilien Pecoul @ouarzy



Disclaimer



- We're not mathematicians
- Just curious programmers



Question?

www.slido.com

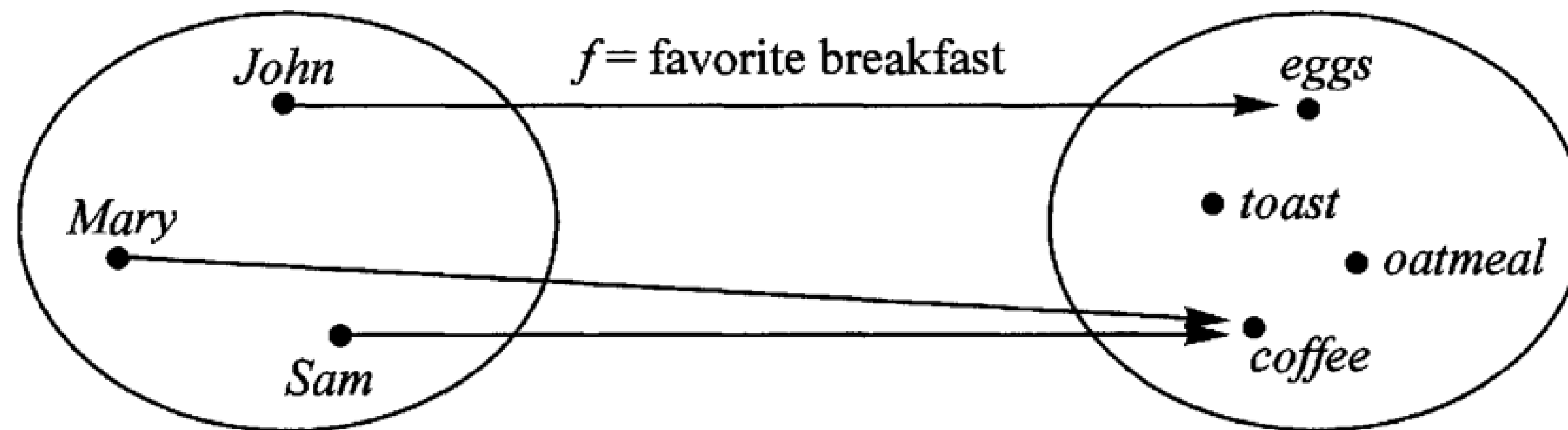


Why Category Theory ?

Common abstract language

Behaviors analysis instead of object analysis

It's all about structure



Code review

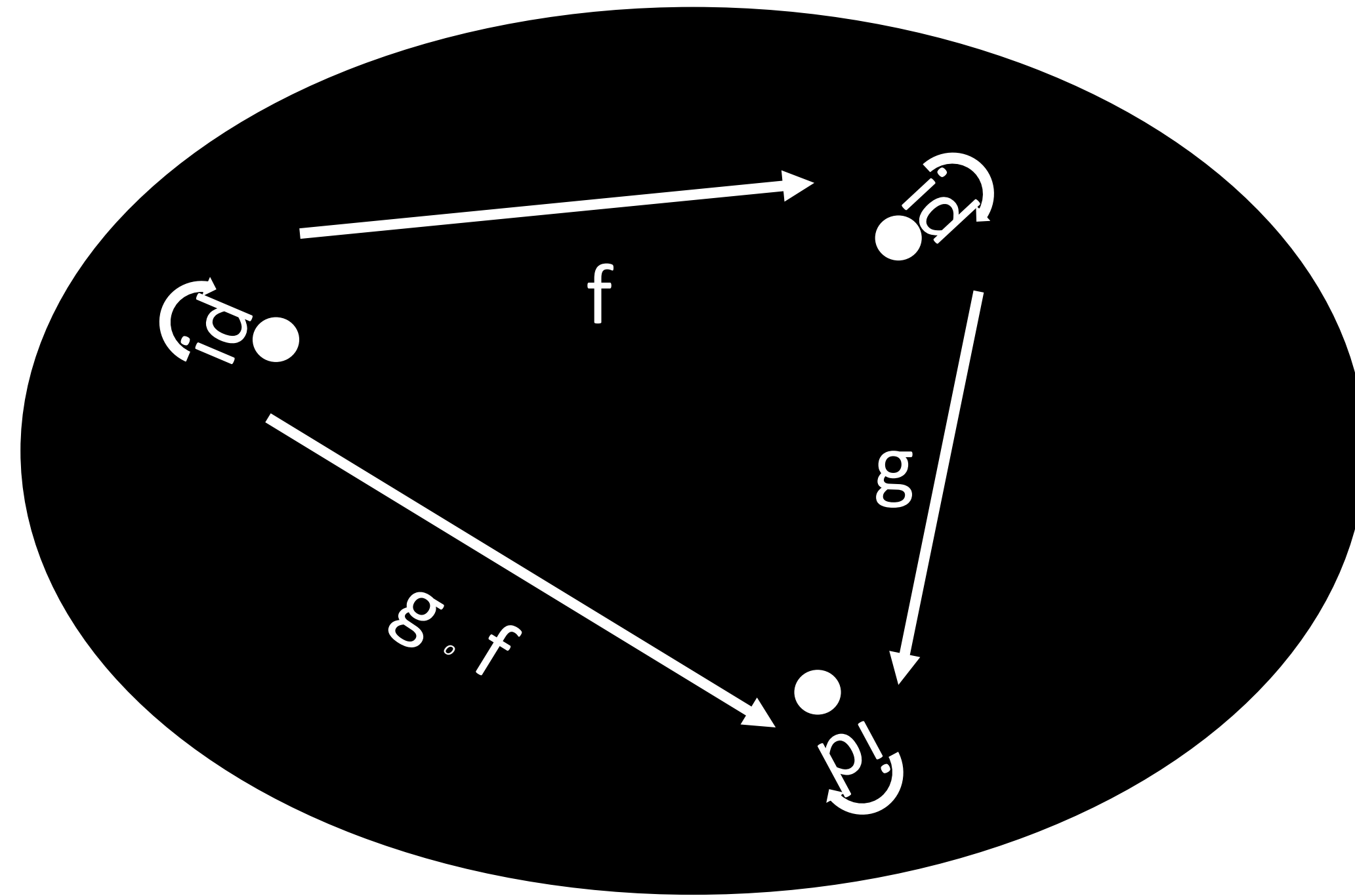
```
Romain BERTHON
public Amount GetTotalAmountOfSuspiciousOperations(IReadOnlyList<AccountLine> lines)
{
    var suspiciousOperations : IReadOnlyList<AccountLine> = GetSuspiciousOperations(lines);
    return GetTotalAmount(suspiciousOperations); // # 1 Composition
}

1 usage Romain BERTHON
public IReadOnlyList<AccountLine> GetSuspiciousOperations(IReadOnlyList<AccountLine> lines) =>
    lines // IReadOnlyList<AccountLine>
        .Select(line => (line, isSuspicious: IsSuspiciousAmount(line))) // # 3 Functor & map
                                                // Product
        .SelectMany(x : (line, isSuspicious) => x.isSuspicious // Where
            ? new List<AccountLine> { x.line } // Monad & Bind
            : new List<AccountLine>()) // IEnumerable<AccountLine>
        .ToList(); // # 1 Composition

1 usage Romain BERTHON
private static bool IsSuspiciousAmount(AccountLine line) => // # 2 Morphisms: Loss of information
    line.Amount.Value > 10_000m;

1 usage Romain BERTHON
public Amount GetTotalAmount(IReadOnlyList<AccountLine> lines) =>
    lines // IReadOnlyList<AccountLine>
        .Select(line => line.Amount) // # 3 Functor & map
        .Aggregate(Amount.Zero, Amount.Add); // Monoid
}
```


What's Category Theory ?

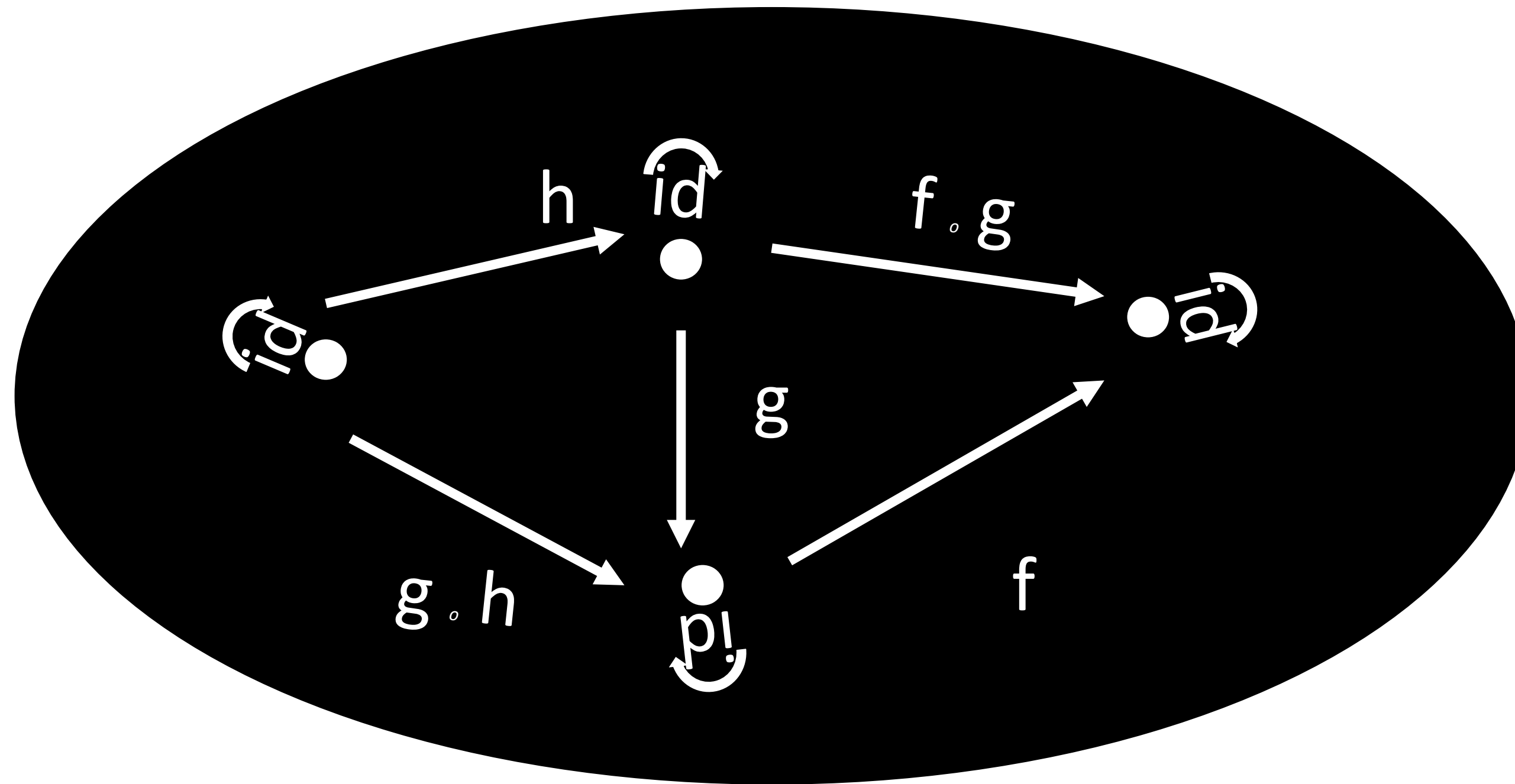


A bunch of **morphisms**, they start and finish with an **object**

Identity: $(a \rightarrow a)$

Composition: $(a \rightarrow b) \rightarrow (b \rightarrow c) = (a \rightarrow c)$

What's Category Theory ?



Identity law: $(\text{id} \circ f) = (f \circ \text{id}) = f$

Associative law: $(f \circ g) \circ h = f \circ (g \circ h)$

Essence of Category Theory ?

Composition

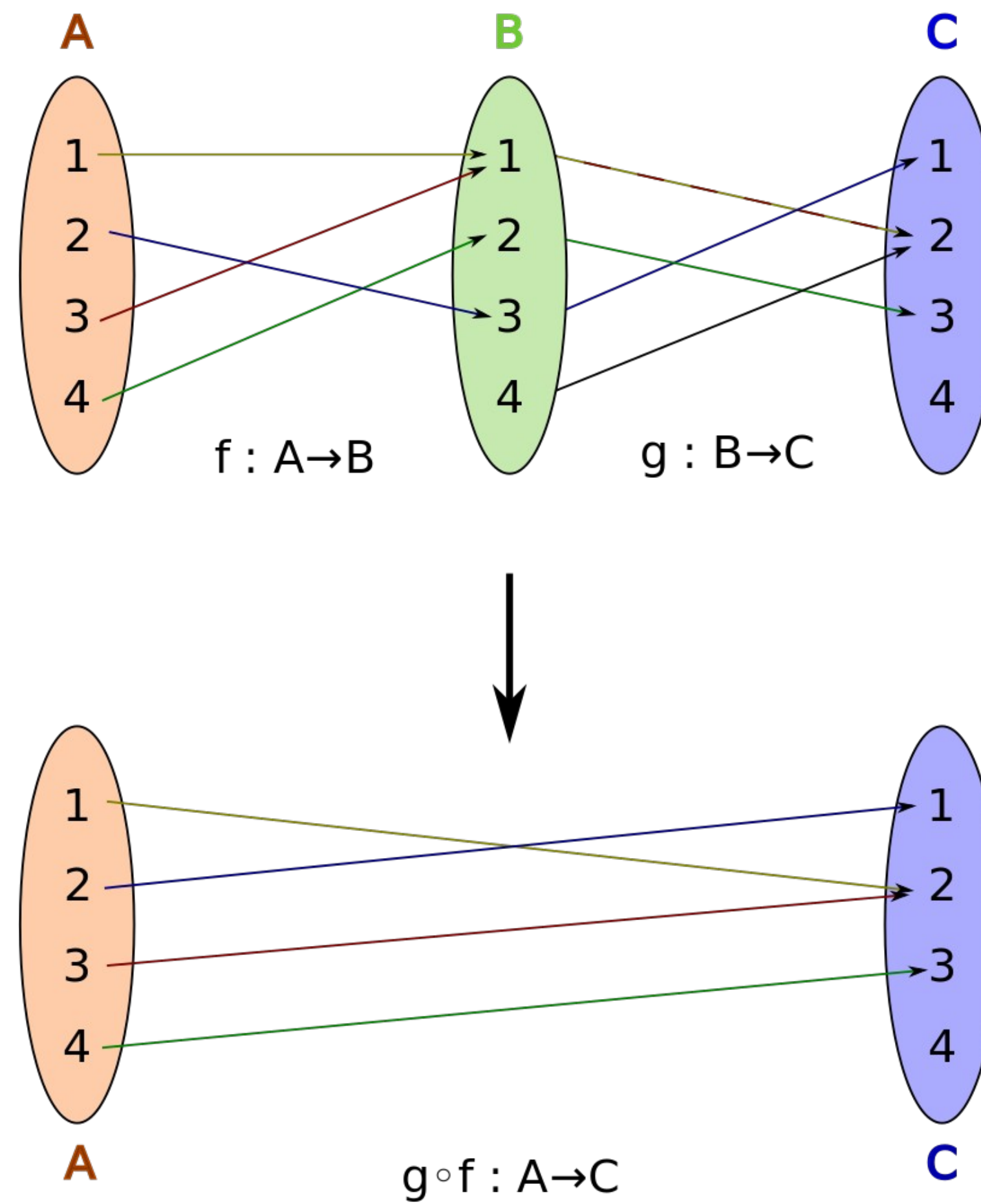
Identity

Abstraction

(Functionnal)

Programmers know it!

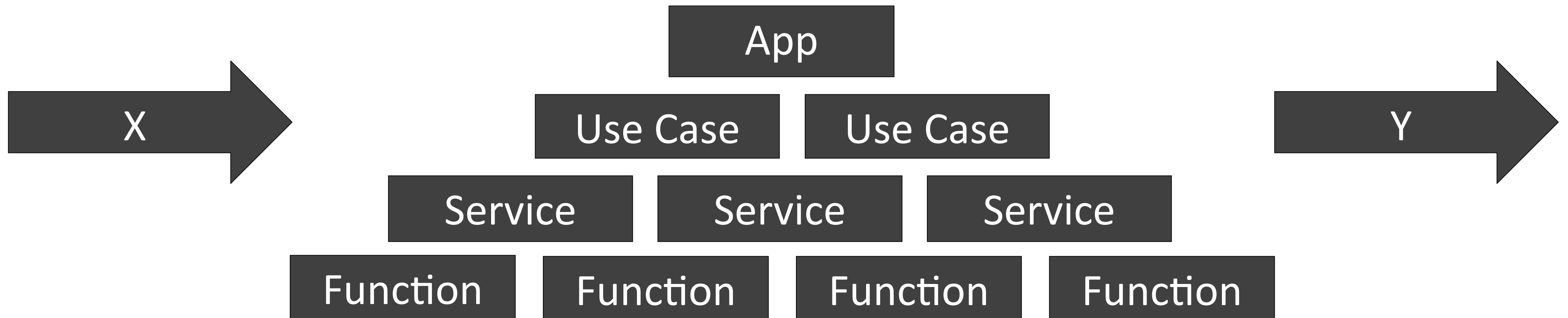
Composition



Composition

Fundamental in functional programming:

- The result of each function is the argument of the next
 - The result of the last one is the result of the whole



Composition

Chunk big problems into smaller ones

Requires no side effect



Example ?

```
Romain BERTHON
public class Composition
{
    1 usage Romain BERTHON
    private int f(decimal value) => (int)value;

    1 usage Romain BERTHON
    private bool g(int value) => value % 2 == 0;

    Romain BERTHON
    public bool g_after_f(decimal value) => g(f(value));
}
```

```
decimal -> int Romain BERTHON
let f (x: decimal) = int x

int -> bool Romain BERTHON
let g (x: int) = x % 2 == 0

decimal -> bool Romain BERTHON
let g_after_f = f >> g
```

Composition

Composition allows to solve big problems with little composed solutions

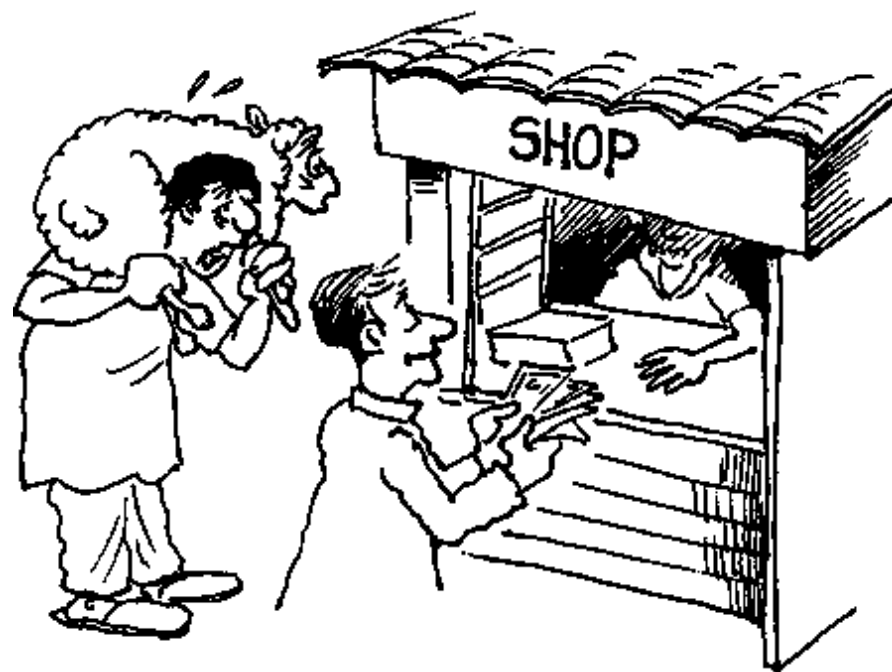


We need composition because reality is too hard to deal with

Abstraction

How do we build software ?

Reality → Software Model → Code



Product

Money

Person

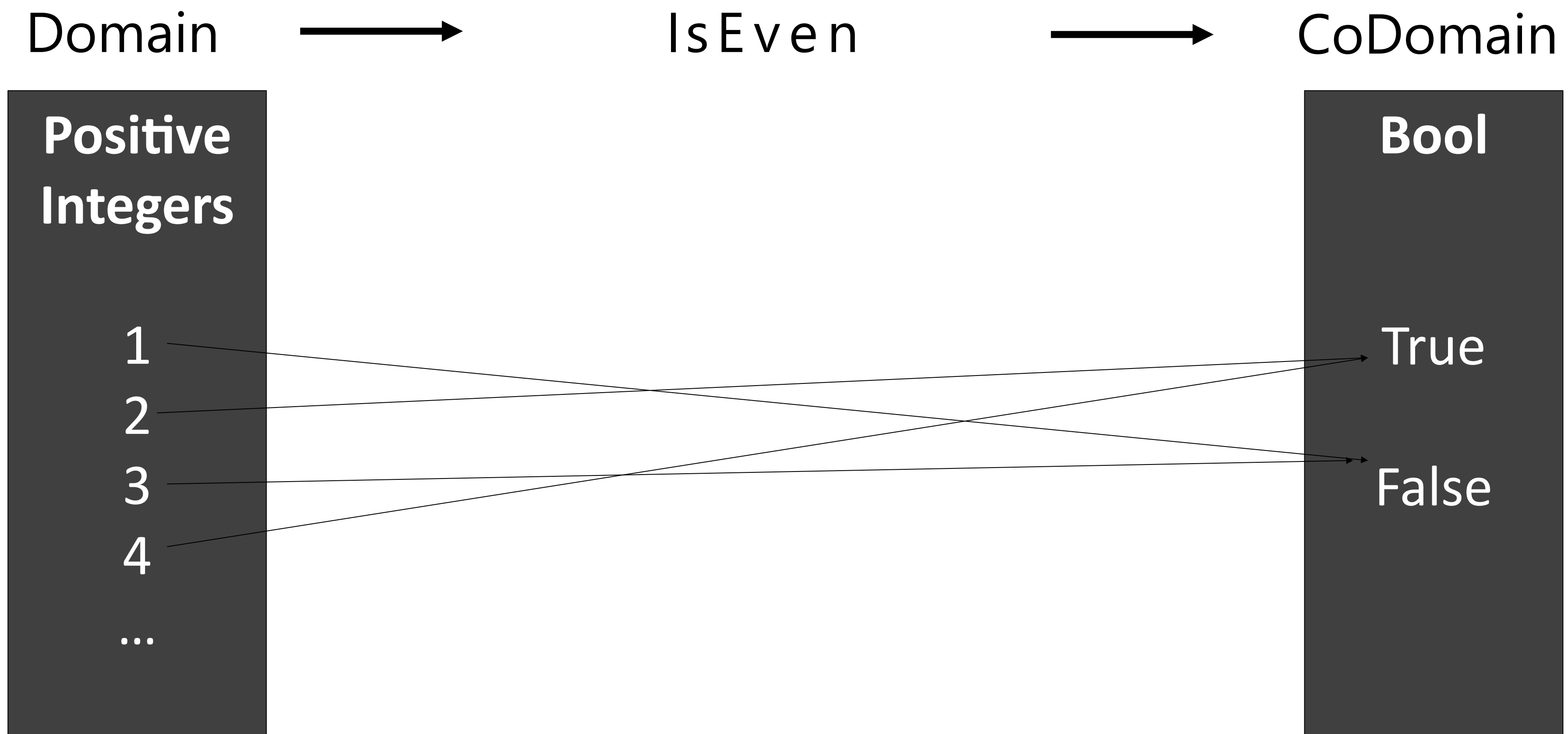
Seller

Buyer

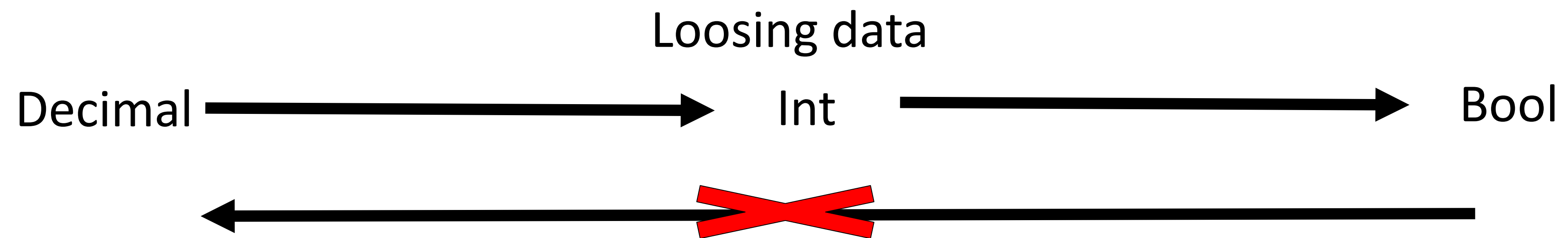
```
class Checkout {  
    private double tax;  
  
    public Checkout(double tax) {  
        this.tax = tax;  
    }  
  
    double total(final List<Product> products) {  
        return products  
            .stream()  
            .mapToDouble(Product::getPrice)  
            .sum() * tax;  
    }  
}
```


Abstraction

How do we abstract stuff ?



Example ?



```
AmountState EvaluateAmountState(AccountLine line)
```


Abstraction

Abstracting is hiding (loosing) useless details in order to apply common rules

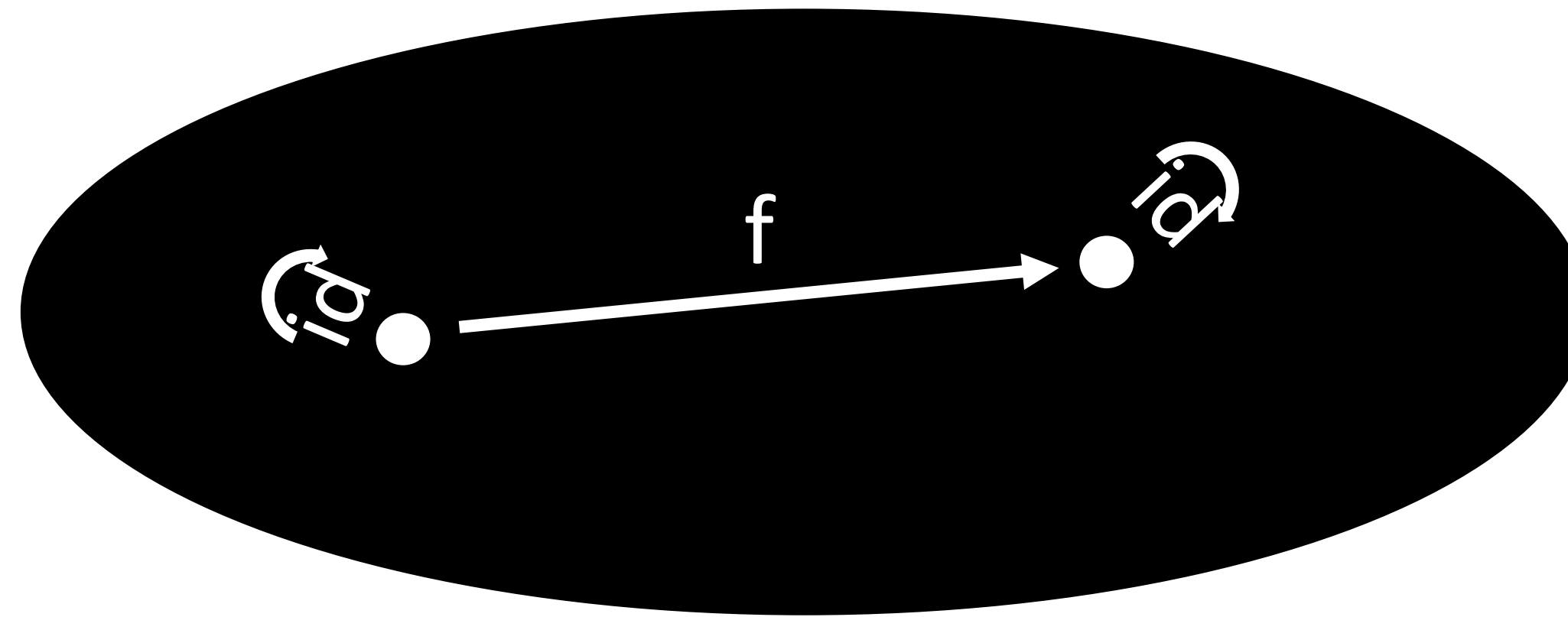


We need composition because reality is too hard to deal with

Identity

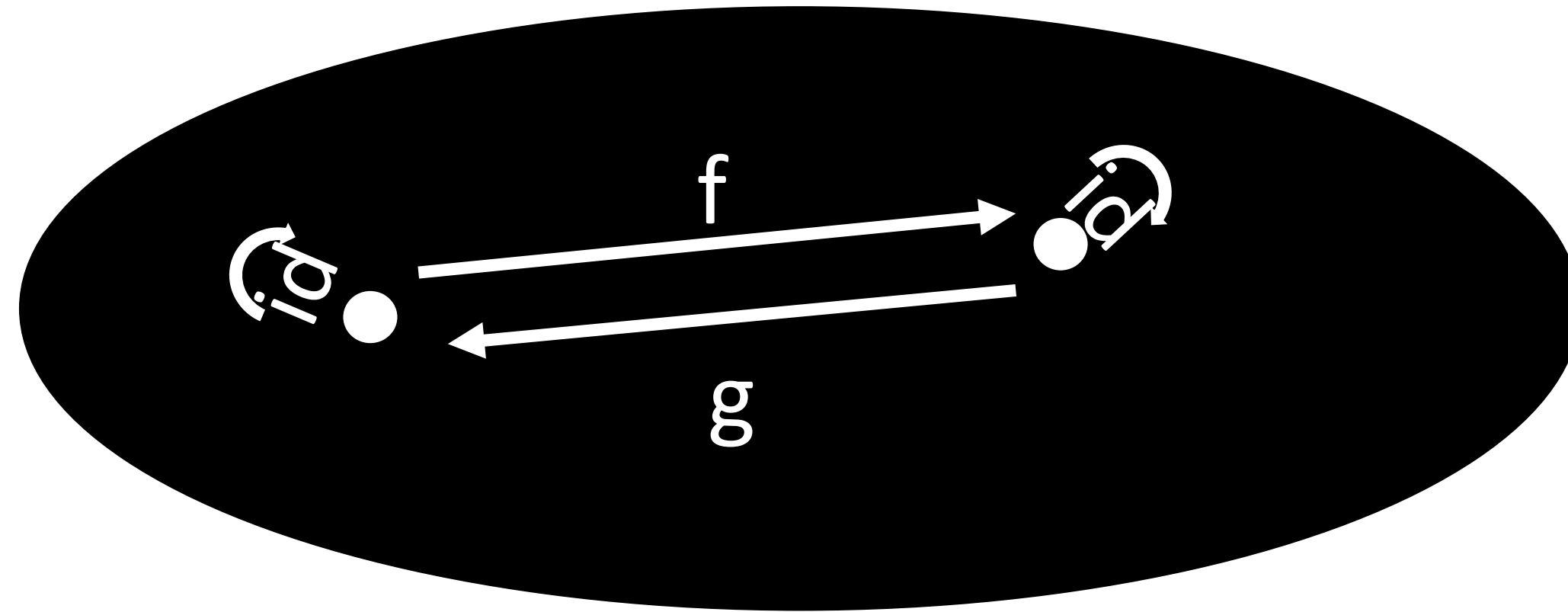


Identity



Identity law: $(\text{id} \circ f) = (f \circ \text{id}) = f$

Identity



Identity law: $(\text{id} \circ f) = (f \circ \text{id}) = f$

$$g \circ f = \text{id}_{\text{Left}}$$

$$f \circ g = \text{id}_{\text{Right}}$$

Identity

Does $(\text{string}, \text{int}) = (\text{int}, \text{string})$?

Isomorphic

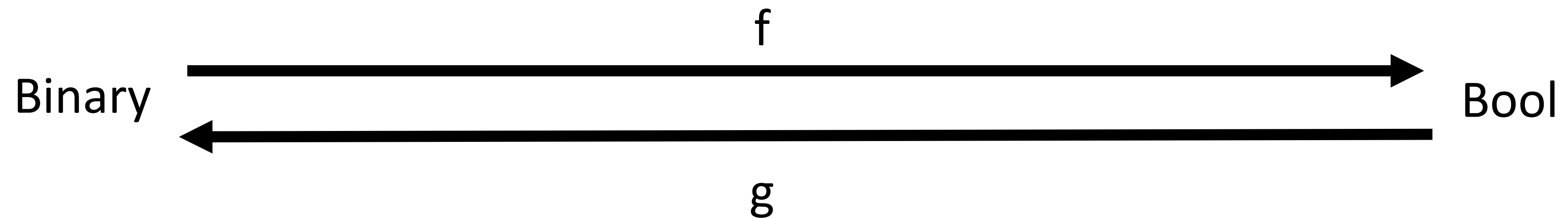
$$\left(\begin{array}{c} \text{f} \\ \circ \\ \text{Id} \end{array} \right) (\text{string}, \text{int}) \overset{\text{f}}{\underset{\text{g}}{\cong}} (\text{int}, \text{string}) \left(\begin{array}{c} \text{f} \\ \circ \\ \text{Id} \end{array} \right)$$

$$g \circ f = \text{Id}_{\text{Left}}$$

$$f \circ g = \text{Id}_{\text{Right}}$$

Example ?

Not loosing data

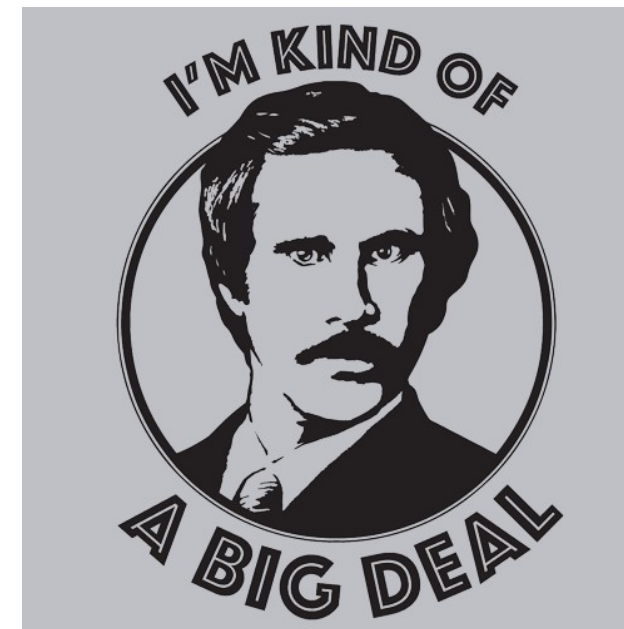


$$f \circ g = \text{IdRight}$$



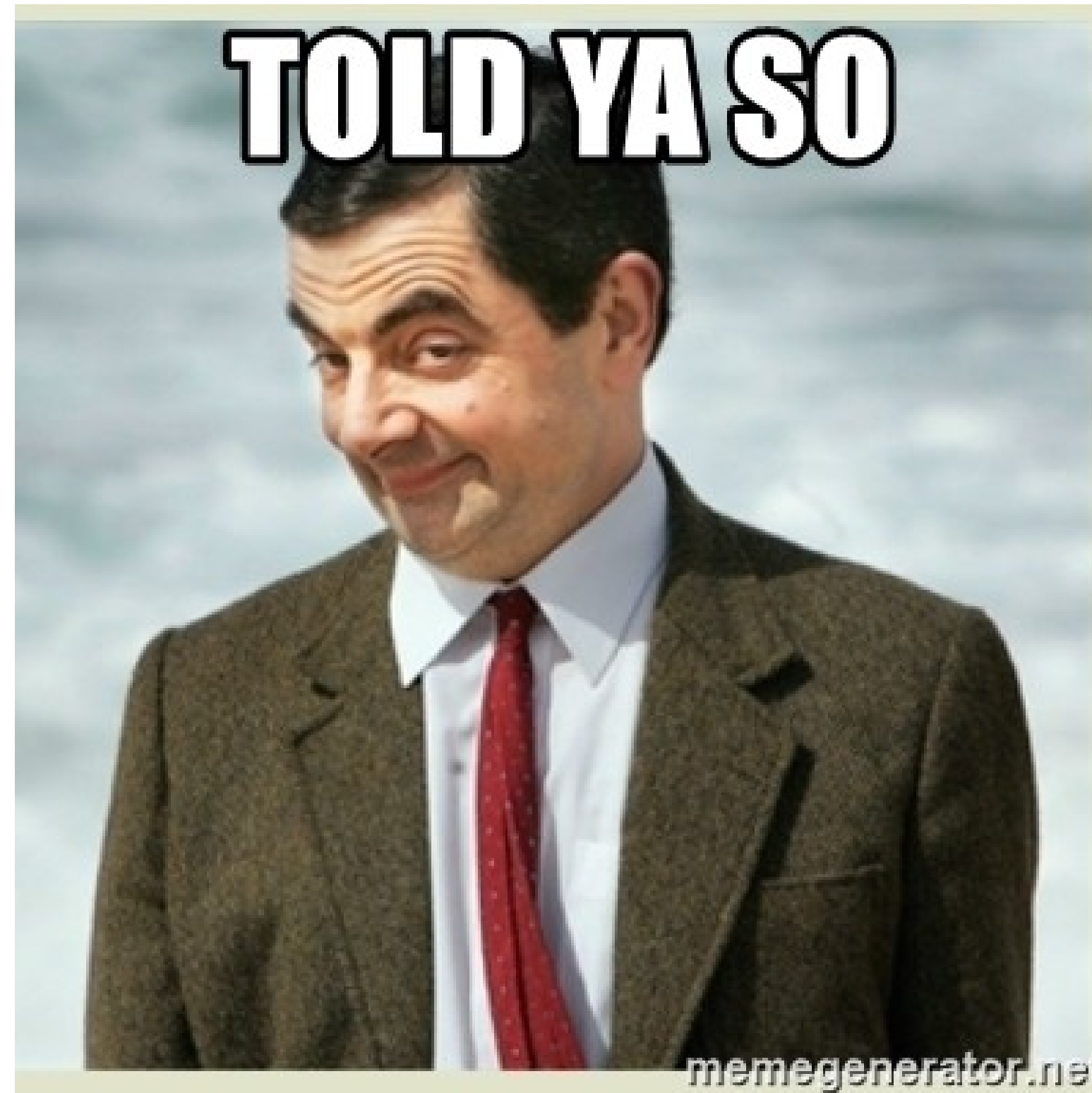
Identity

Identity is important to define an object in a given context

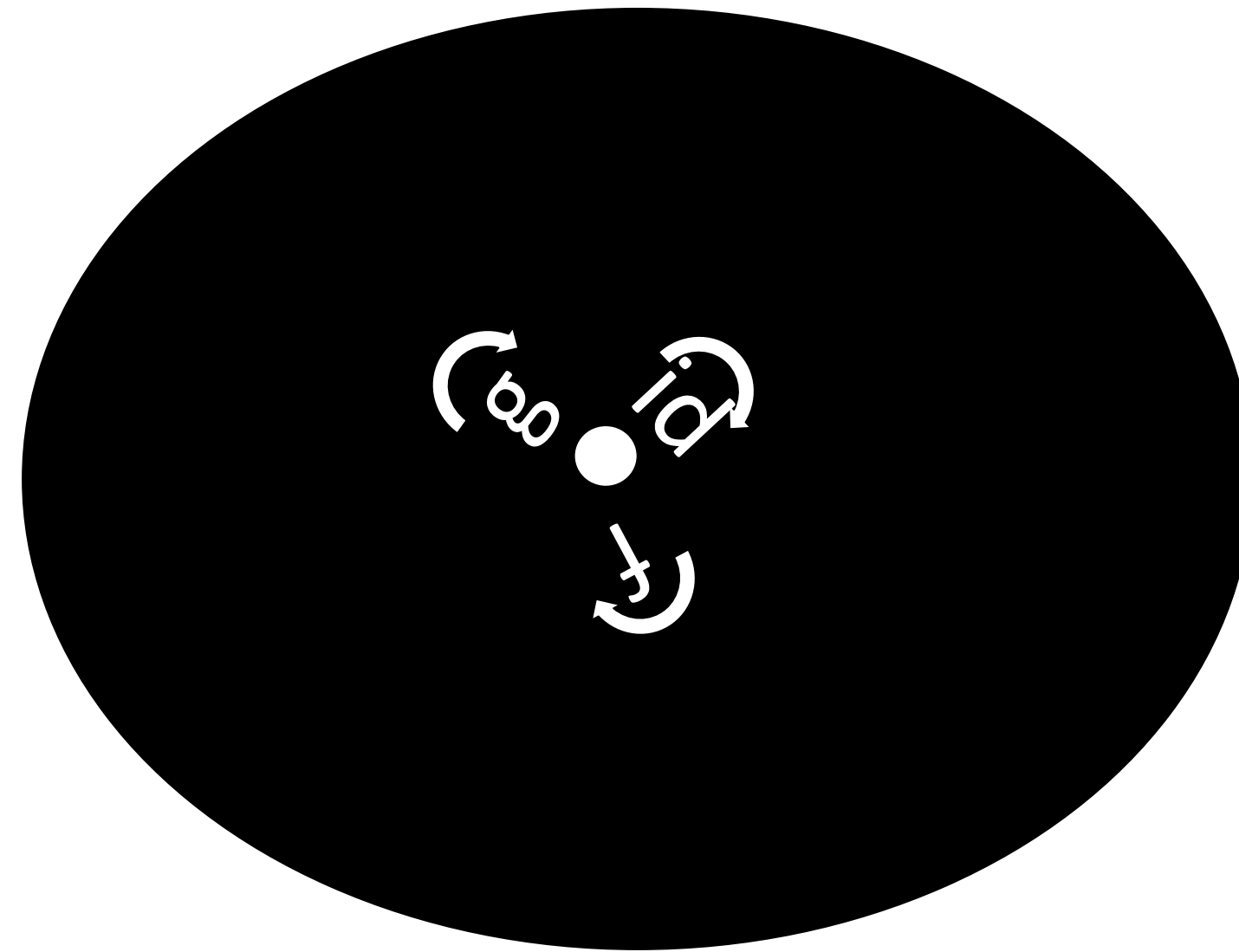


We need identity because reality is too hard to deal with

You already know it!

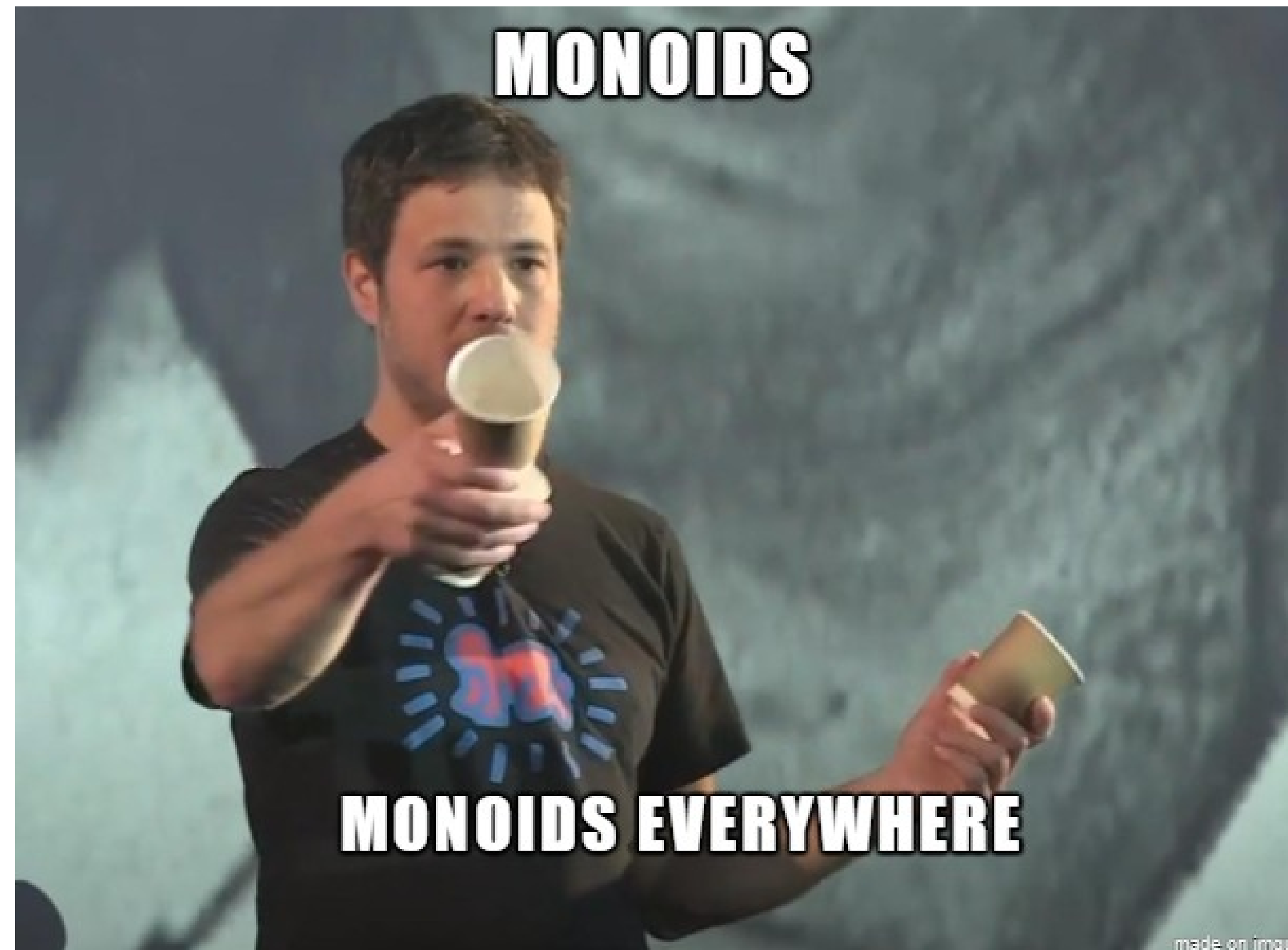


Monoids



A category with only one object

Monoids



<https://youtu.be/J9UwWo2qifg>

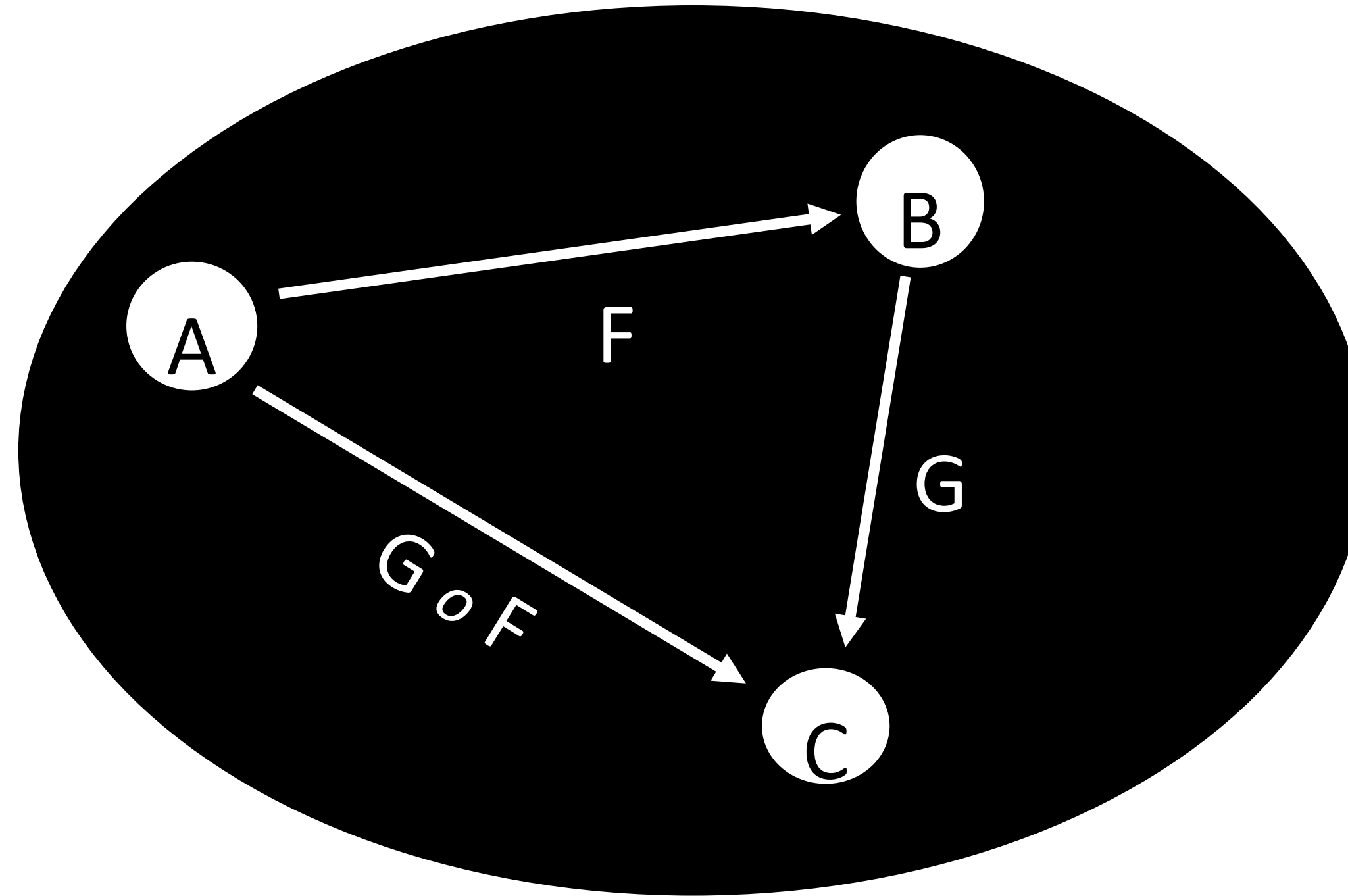
Example ?

```
public record Amount(decimal Value)
{
    public static Amount Add(Amount left, Amount right) => new(left.Value + right.Value);
    public static readonly Amount Zero = new(0m);           // Monoid's Neutral element
}
```

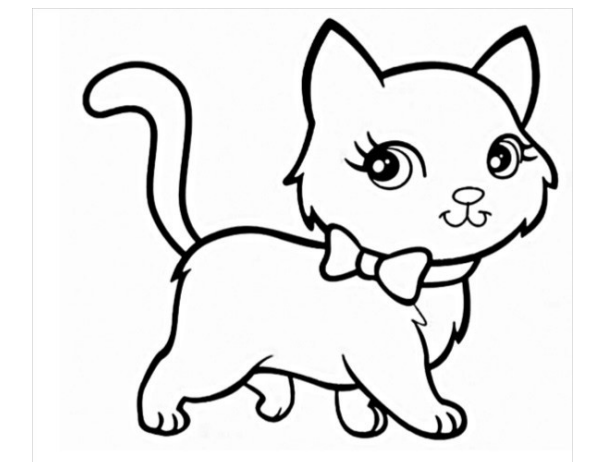
Break



Category you already know



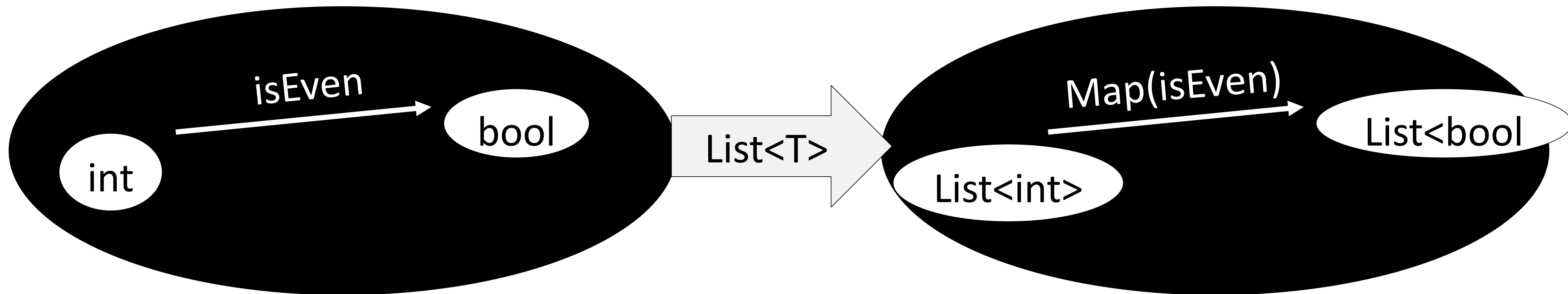
Category of category (cat)



You know category in code

List<T> + map

A functor is a container with a map preserving structure



Example ?

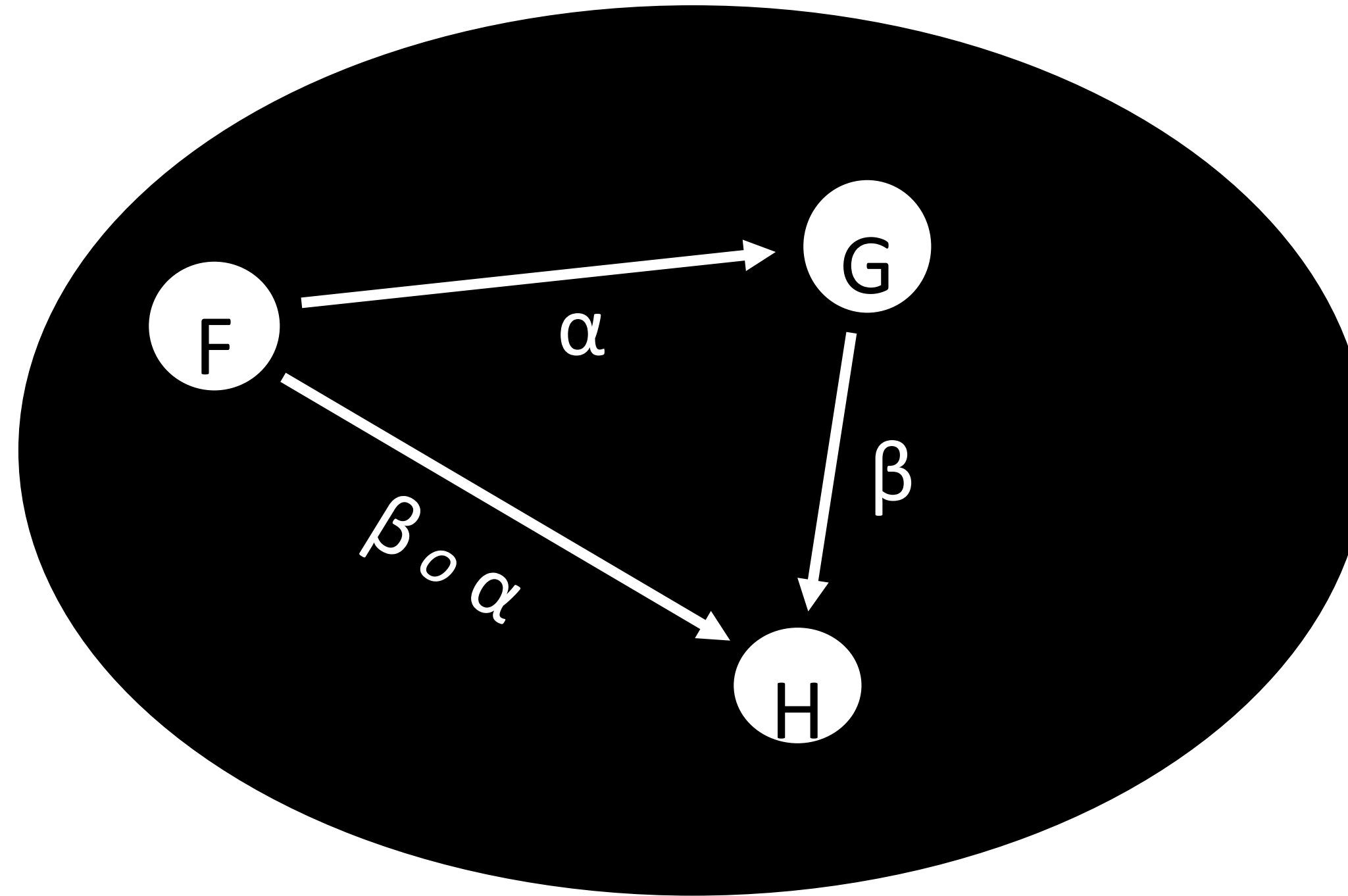
List<T>.map(isEven)



Select in LINQ = map



Category you already know

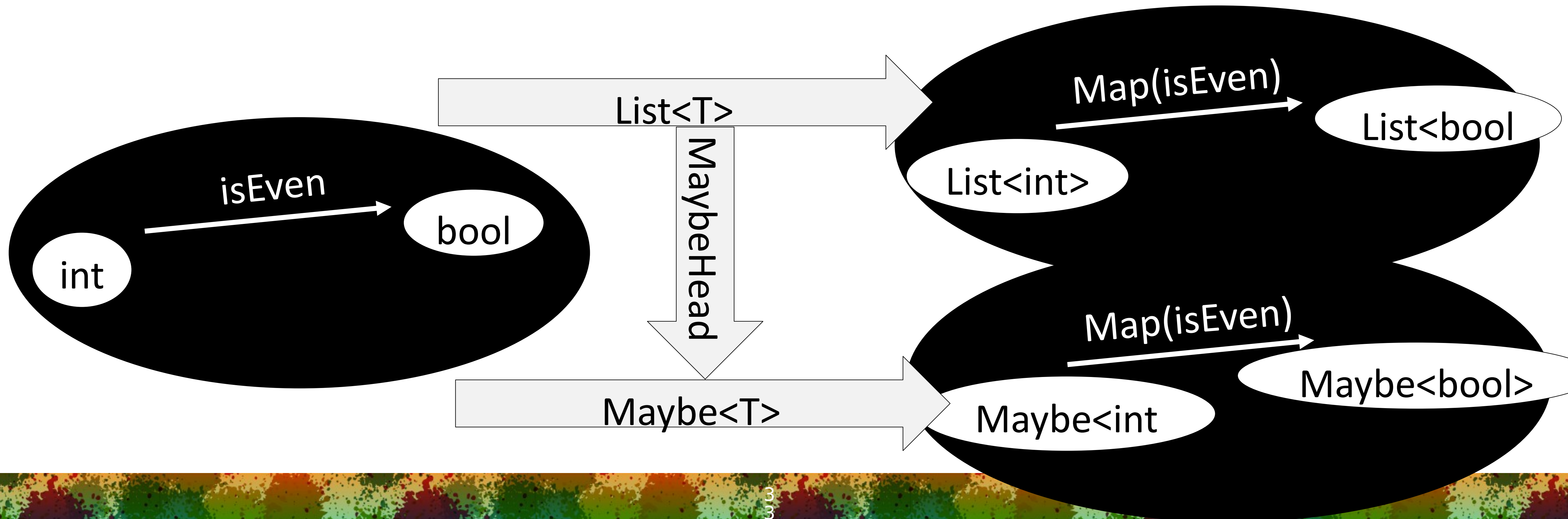


Category of functors

You know category in code

List<T> -> Maybe<T>

A natural transformation is a generic function between two functors



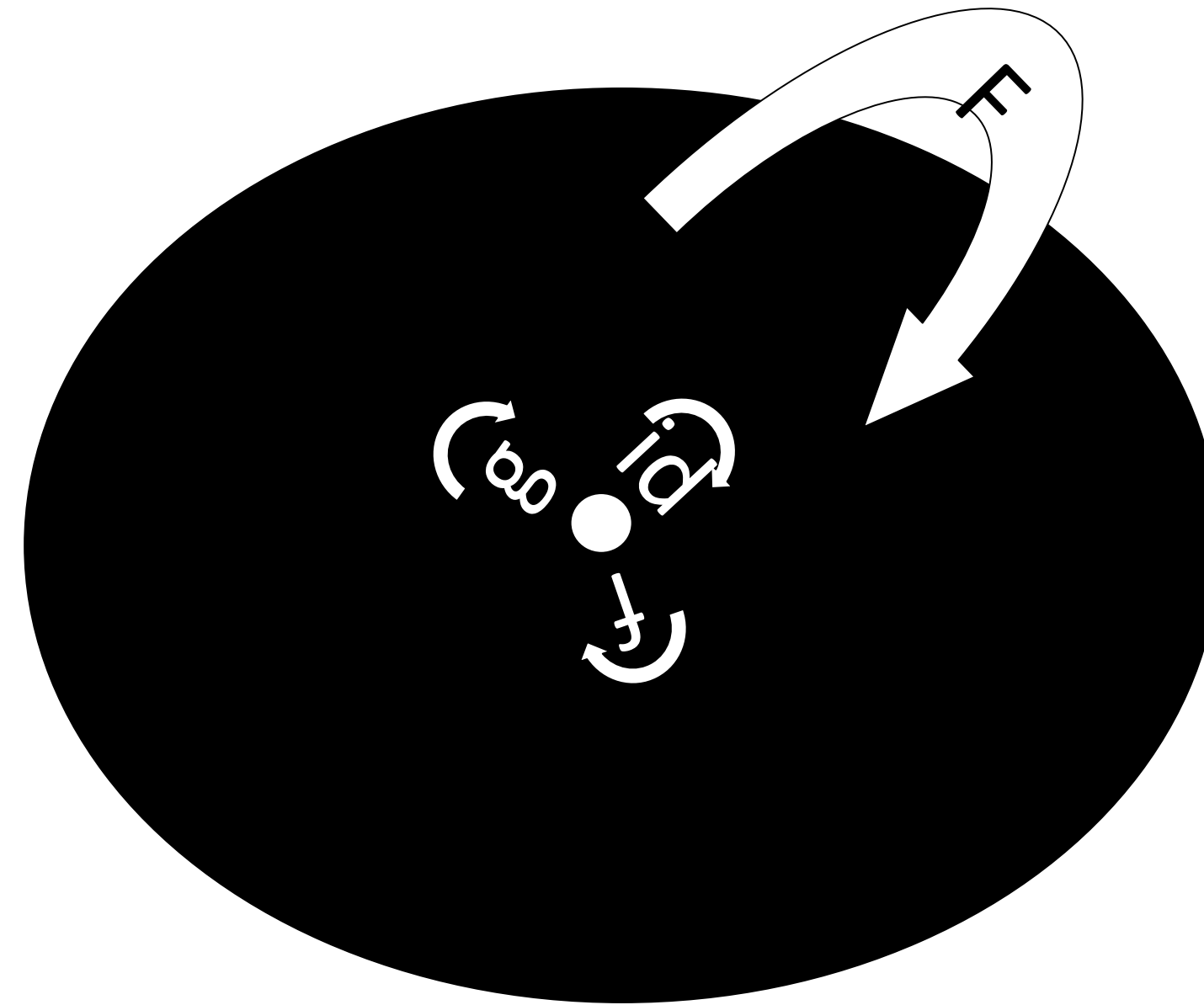
Example ?

List<T>  Maybe<T>
MaybeHead

```
public static Maybe<T> MaybeHead<T>(List<T> list) =>  
    list.Any() ? Maybe<T>.Some(list.First()) : Maybe<T>.None();
```

```
let maybeHead = function  
    | [] -> None  
    | head::tail -> Some head
```

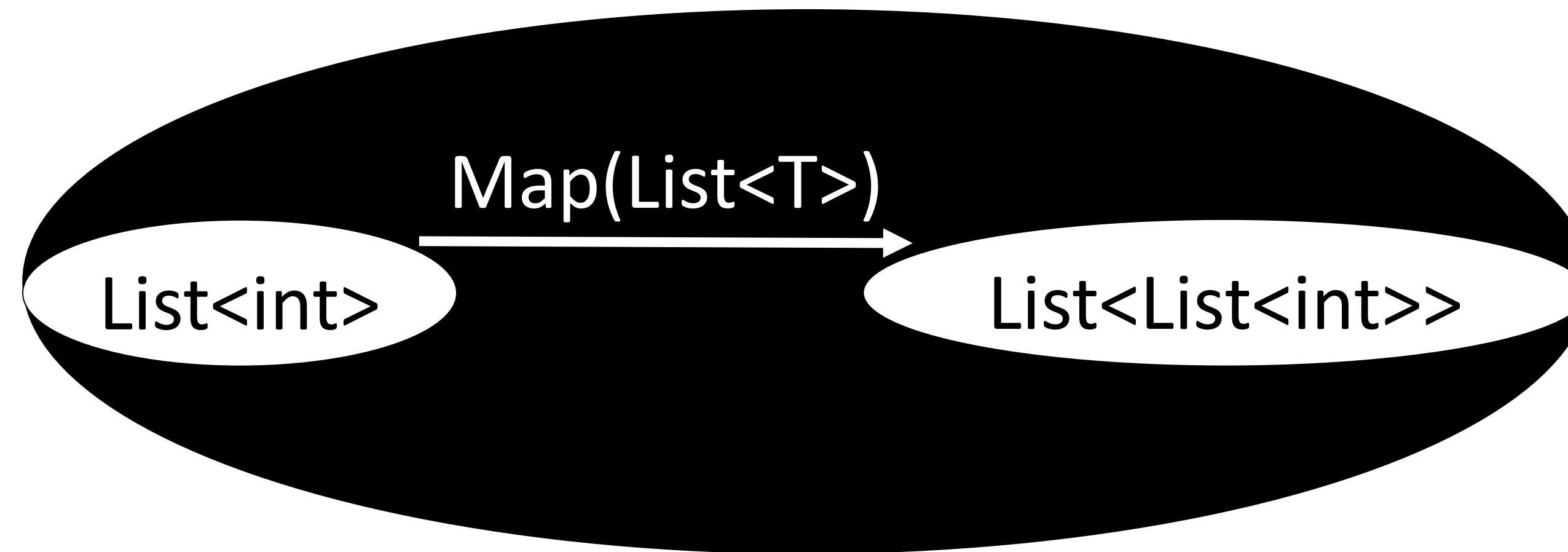

Endofunctor



A functor that map a category to itself

Endofunctors are interesting because they do a good job of representing structures inside categories that work for any object

Endofunctor



A functor that can apply to itself is an endofunctor

"All the Functors we are dealing with in functional programming are Endofunctors" - <https://blog.softwaremill.com/monoid-in-the-category-of-endofunctors-b85bab43587b>

Example ?

Maybe<T> $\xrightarrow{\text{Map(Maybe<T>)}}$ Maybe<Maybe<T>>

1 usage

```
static Maybe<TOut> Map<TValue, TOut>(Maybe<TValue> maybe, Func<TValue, TOut> morphism) =>
    maybe.Match(
        someMorphism: value => Maybe<TOut>.Some(morphism(value)),
        Maybe<TOut>.None); // Maybe<TOut>
```

```
public static Maybe<Maybe<T>> F<T>(Maybe<T> maybe) => Map(maybe, Maybe<T>.Some);
```


Monads



A monad is a monoid in the category of endofunctors

Example ?

```
let safeStringIsPositiveInt = safeStringToInt >> bind safeIsPositive >> map intToString
```


No Silver bullet

Mathematical abstraction are « easy to build »

Abstraction from real world will be harder to build.



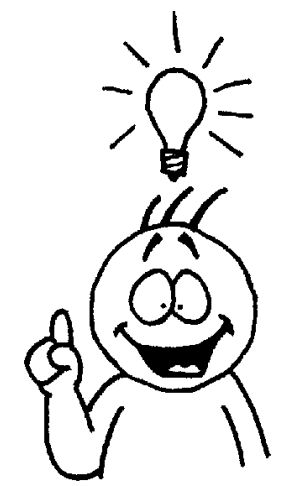
Conclusion

Category theory is not mandatory to code...

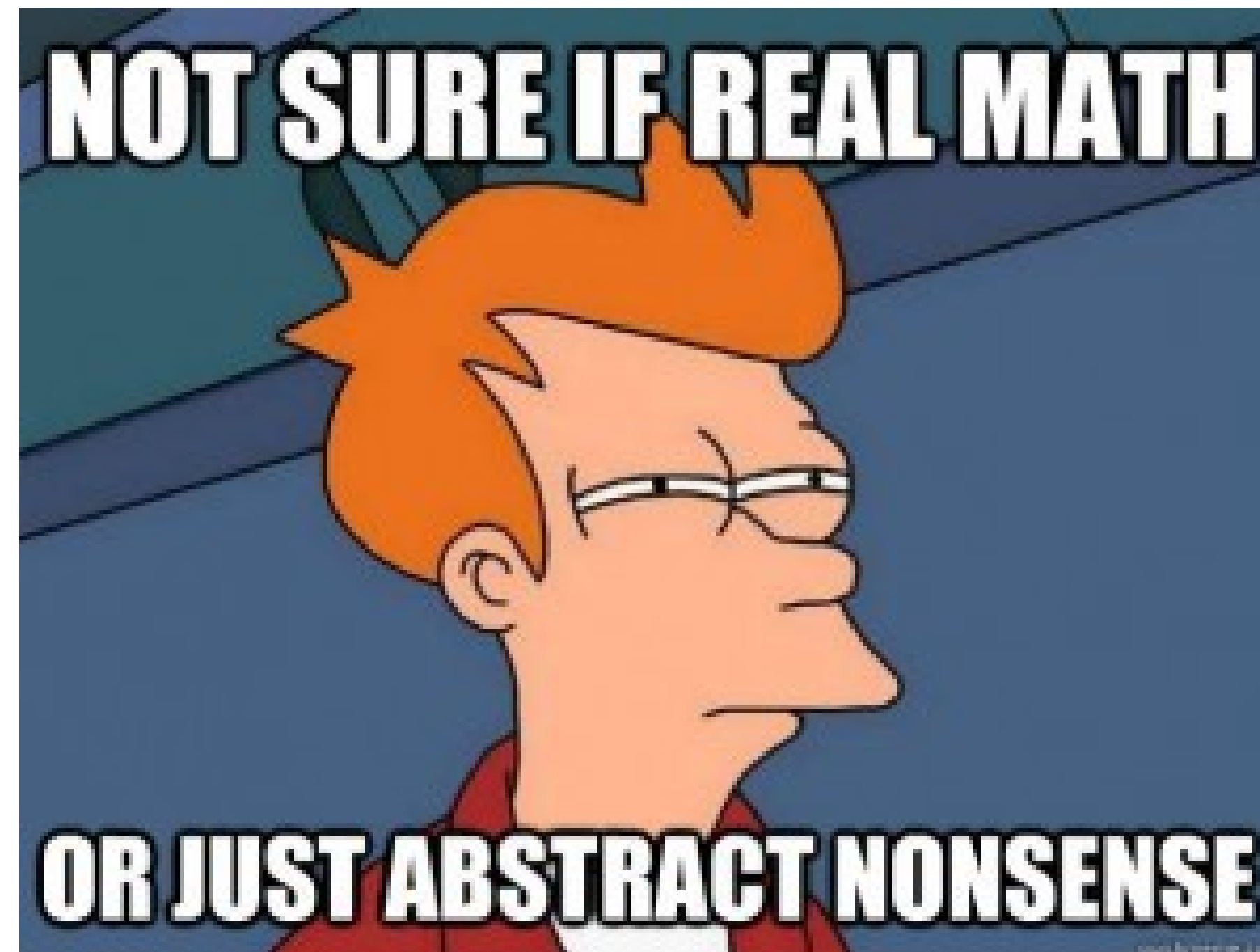
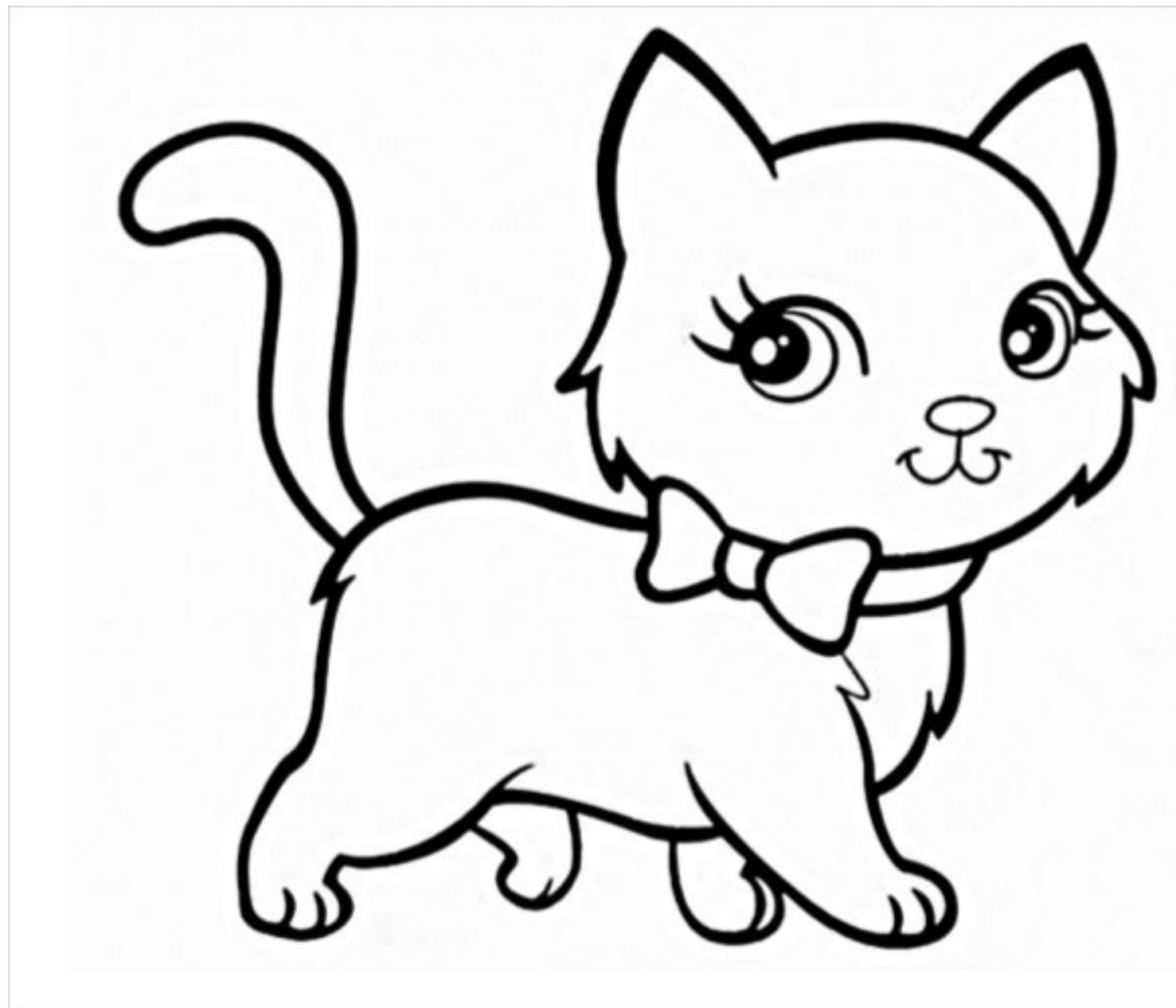
..But it could help to find clever solution to complex programming problems.

[Category theory] does not itself solve hard problems [...] It puts the hard problems in clear relief and makes their solution possible.

—*The Last Mathematician (Hilbert Gottingen)*



Thanks

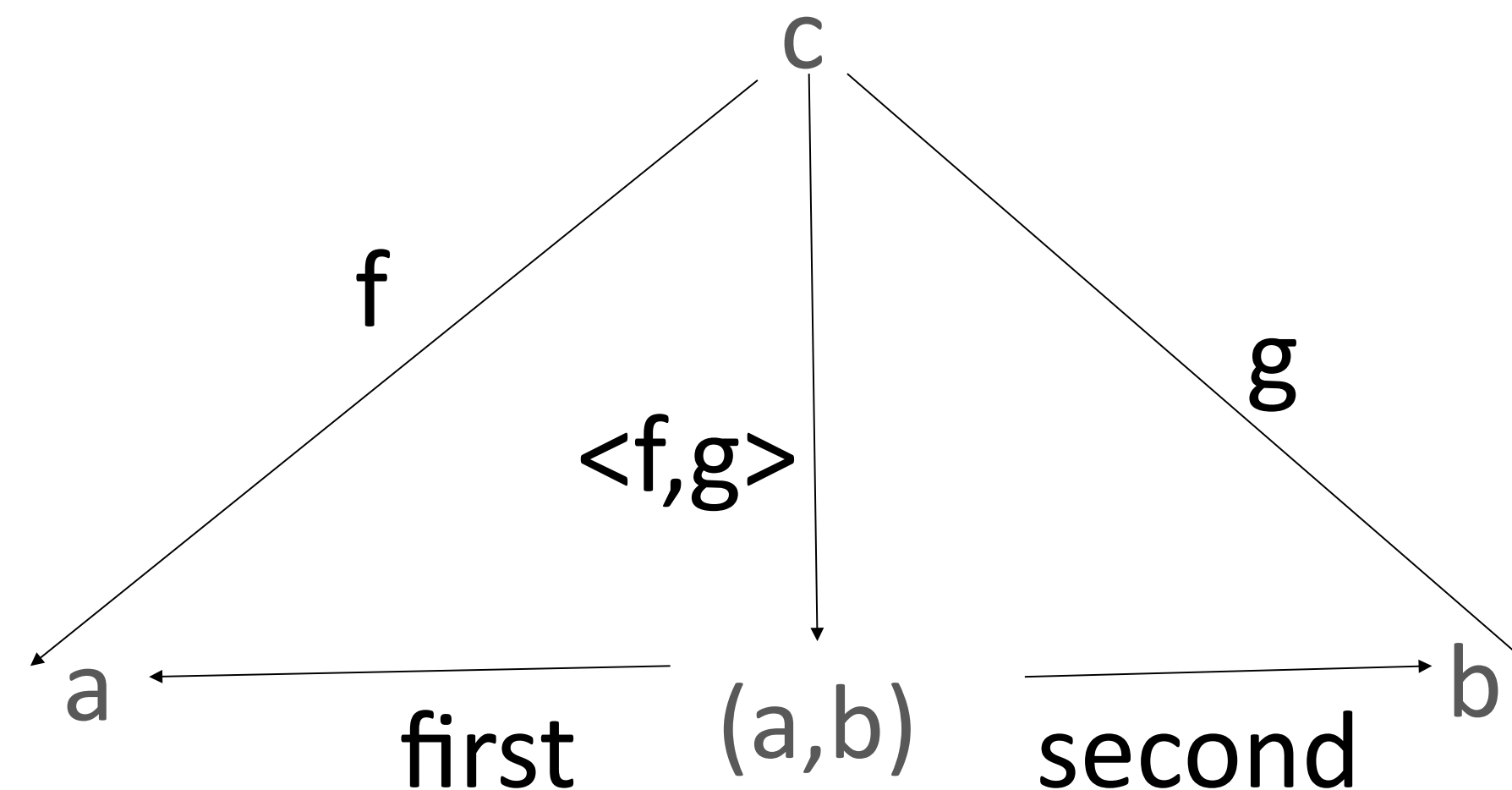


You know category in math

Product

$$A^C * B^C = (A * B)^C$$

$$(C \rightarrow A, C \rightarrow B) = C \rightarrow (A, B)$$

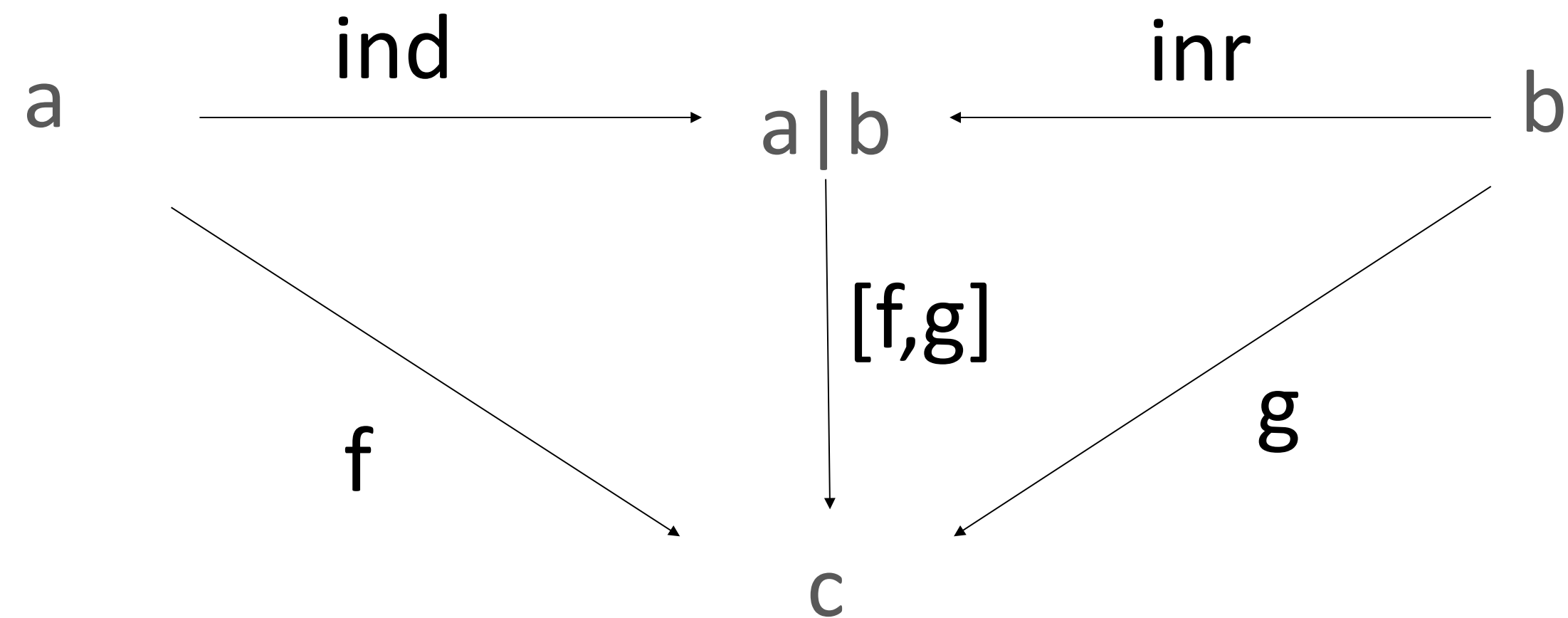


You know category in math

Co Product

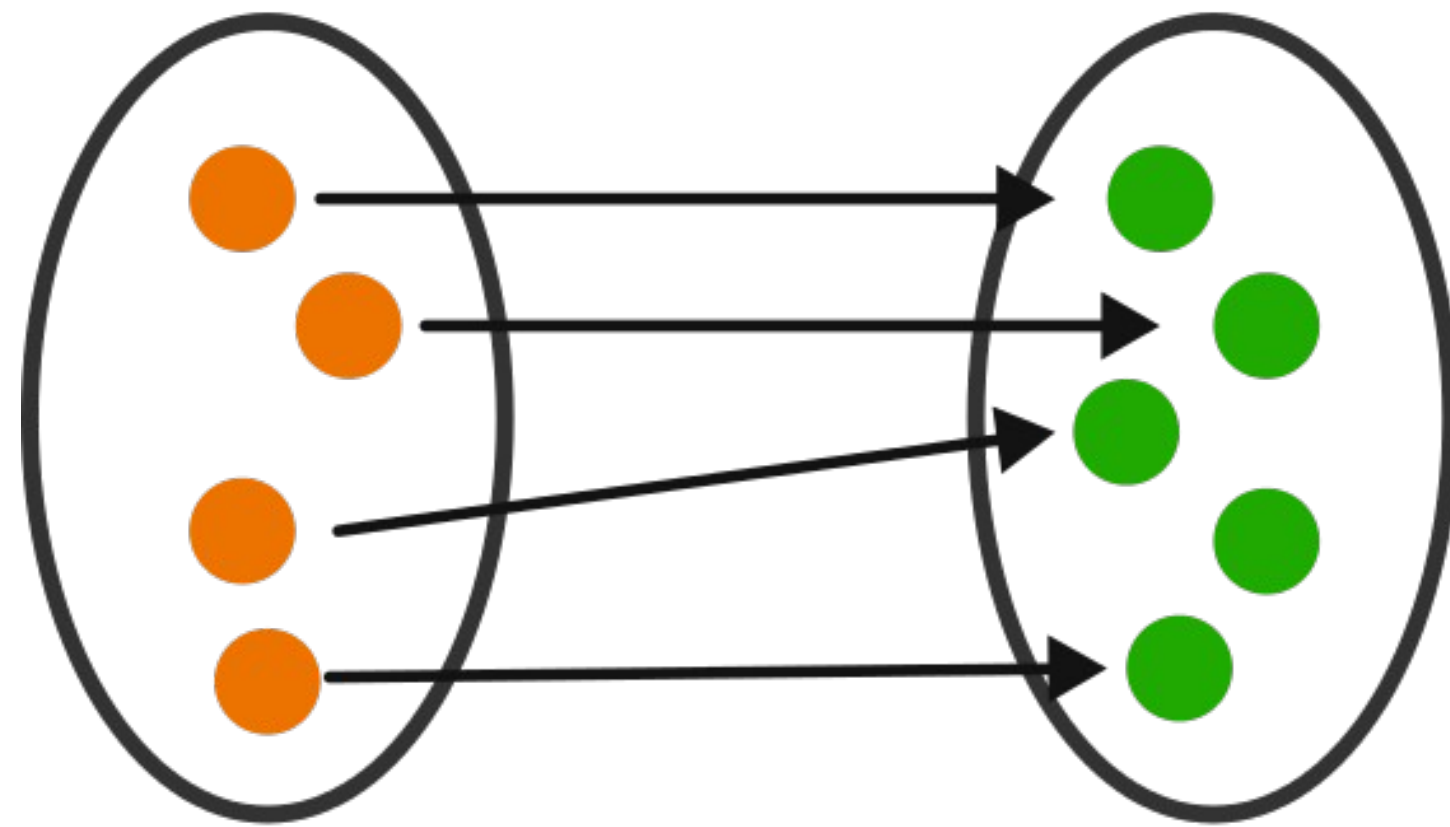
$$(A + B) * C = A * C + B * C$$

$$(A | B, C) = (A, C) | (B, C)$$

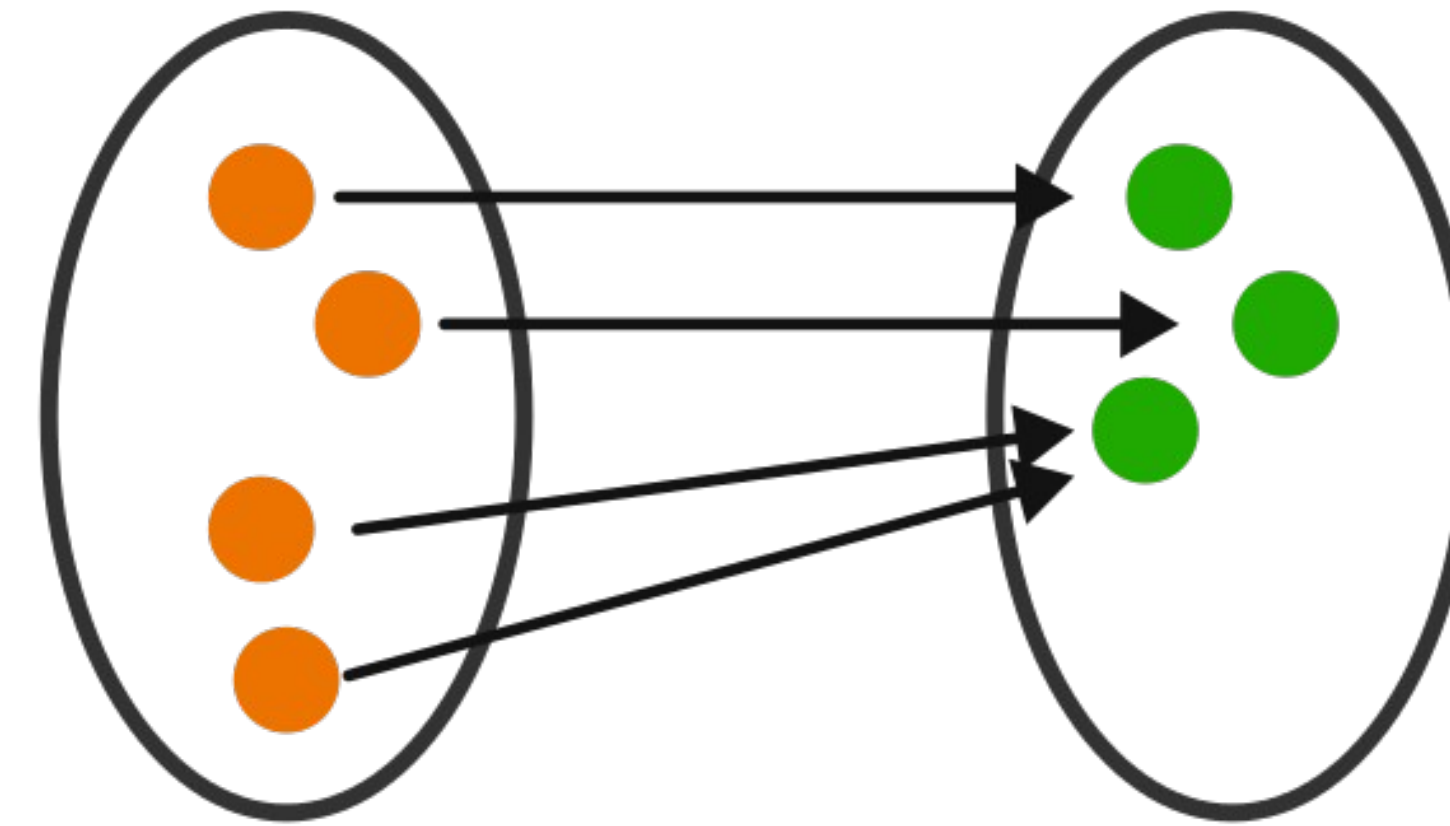


Surjective/Bijective

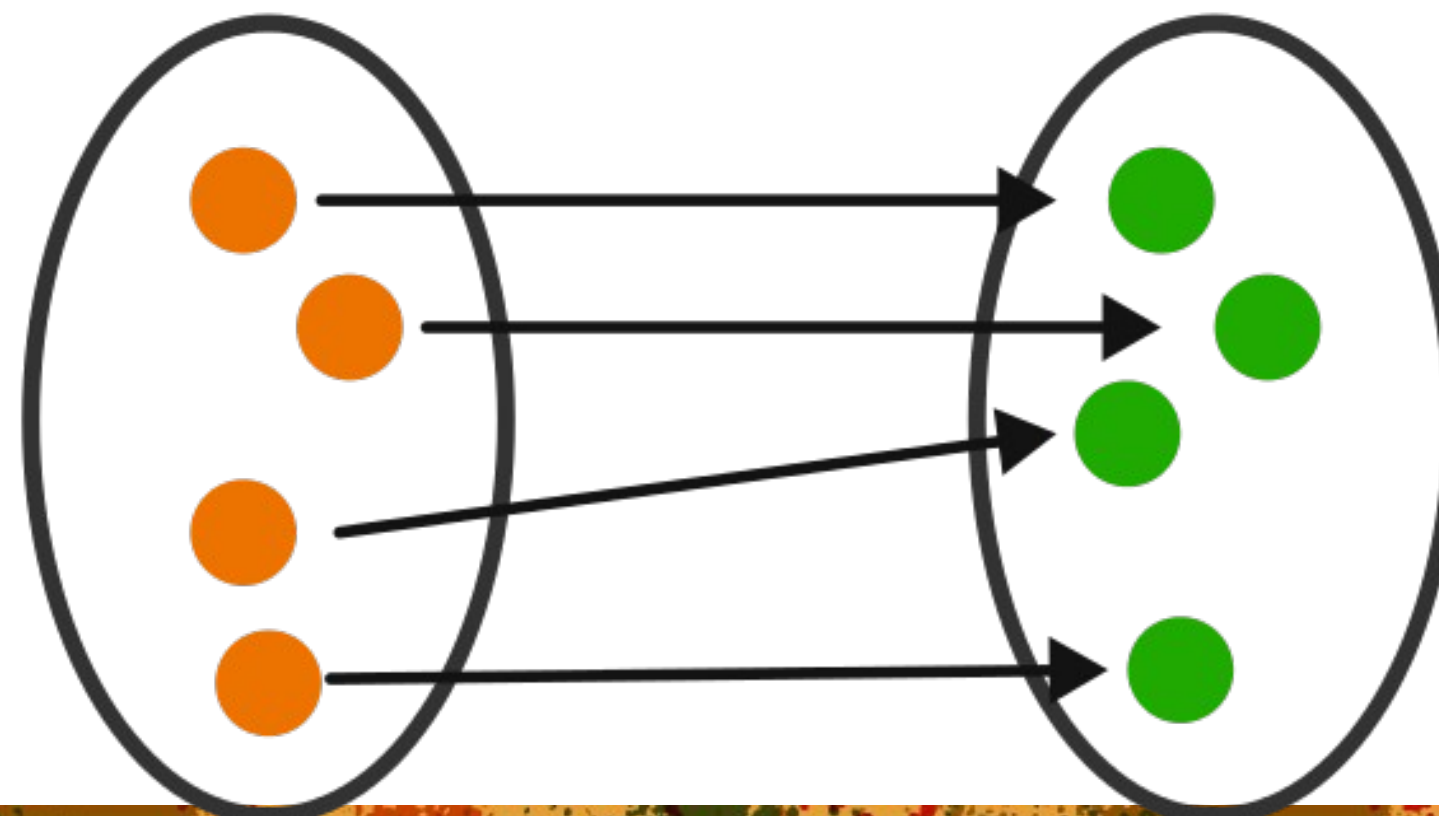
Injection (One-to-One)



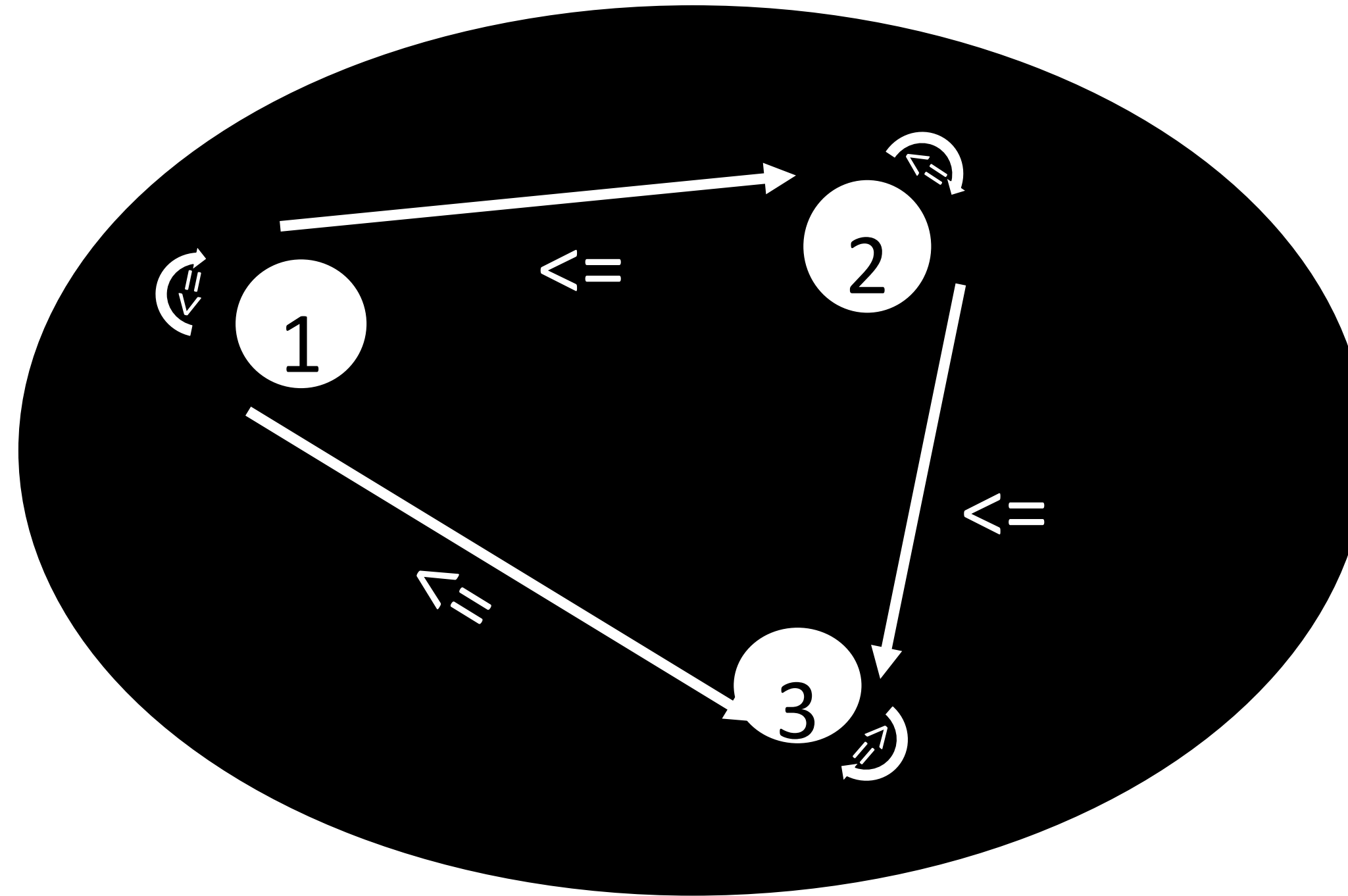
Surjection (Onto)



Bijection (One-to-One and Onto)

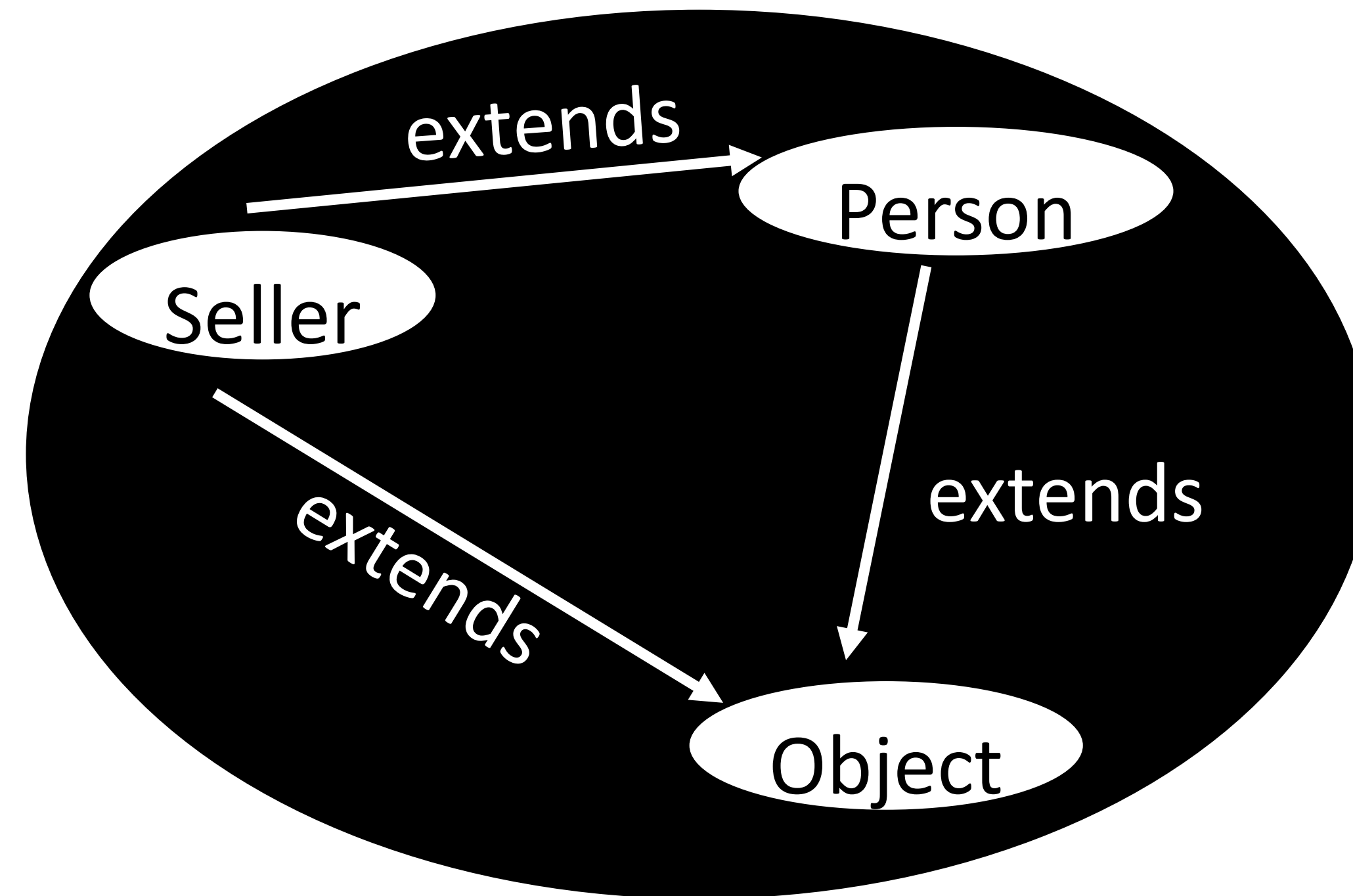


Category you already know



Ordered set

Category you already know



Class hierarchy